FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Crisostomo, Jerome M.
Section:	AN42
Professor:	Ma'am. Crisola Tan
Date:	10/02/2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**

(Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
```

```
}

# -------- Run Model ----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation ----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

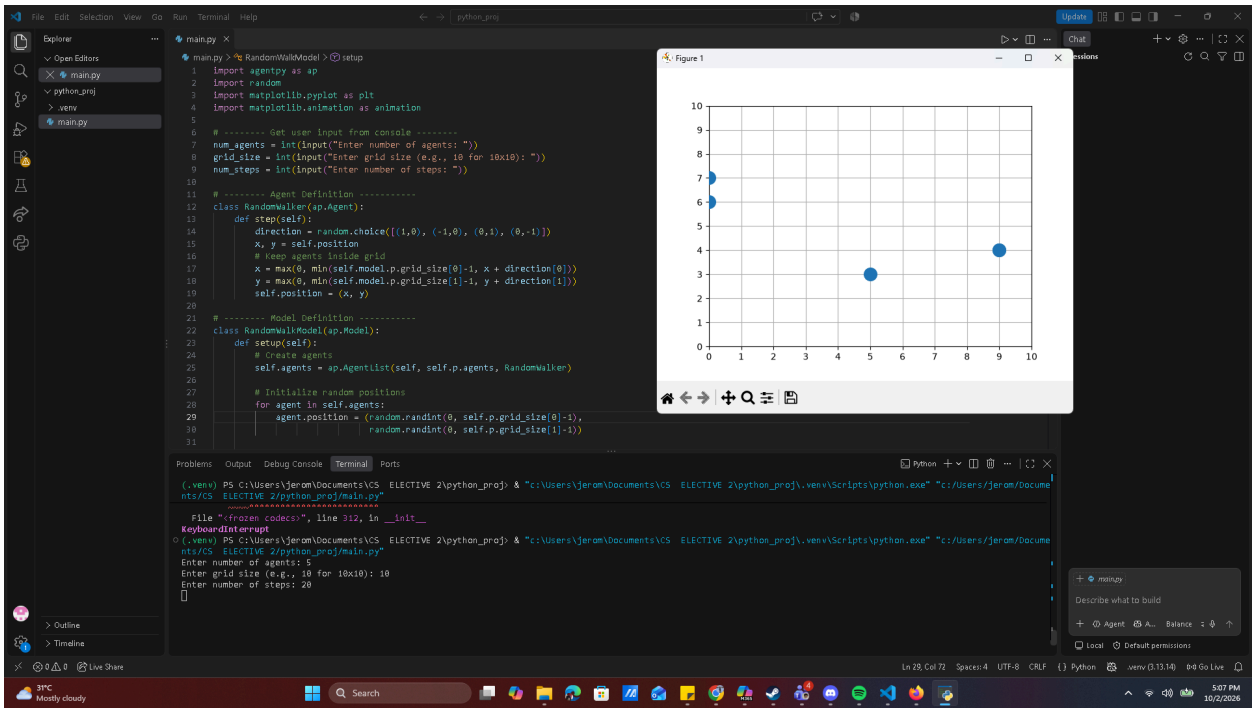
scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.



Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

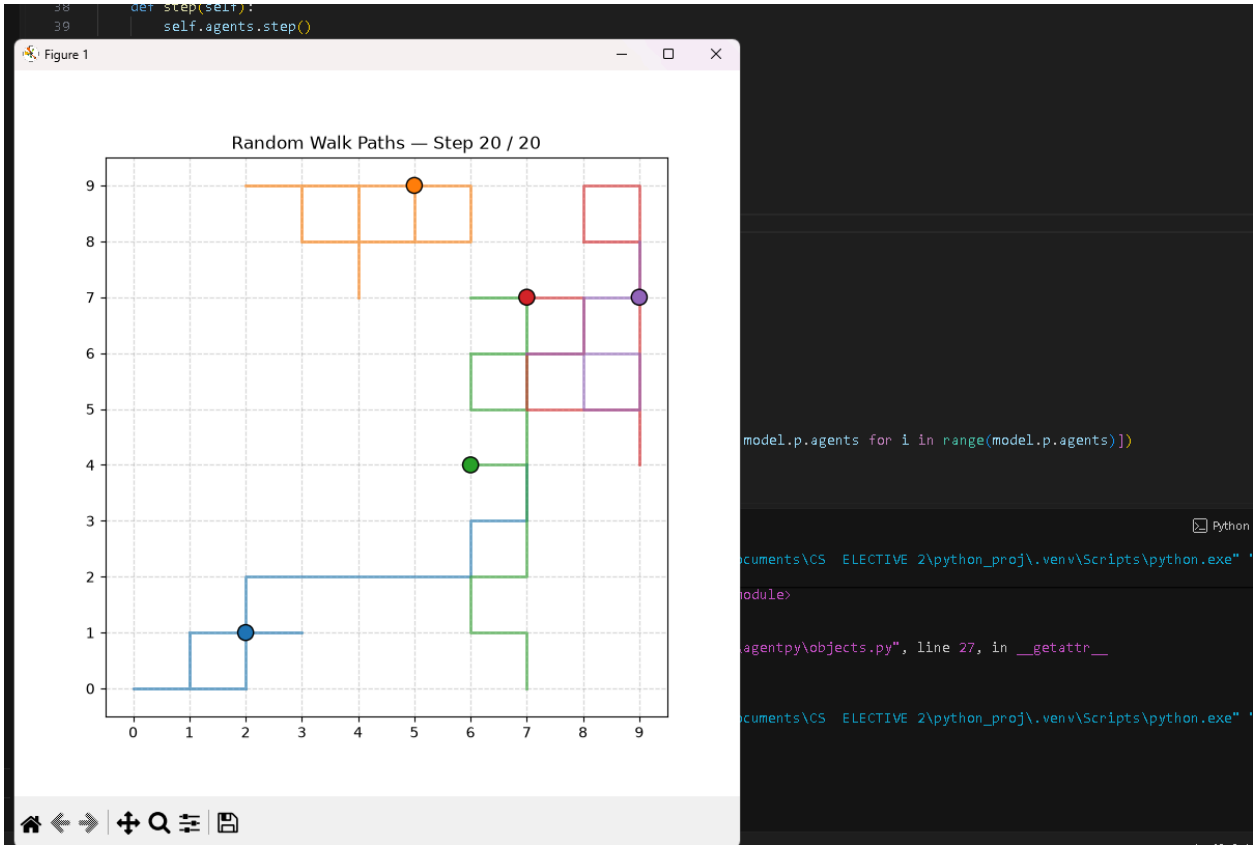
1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

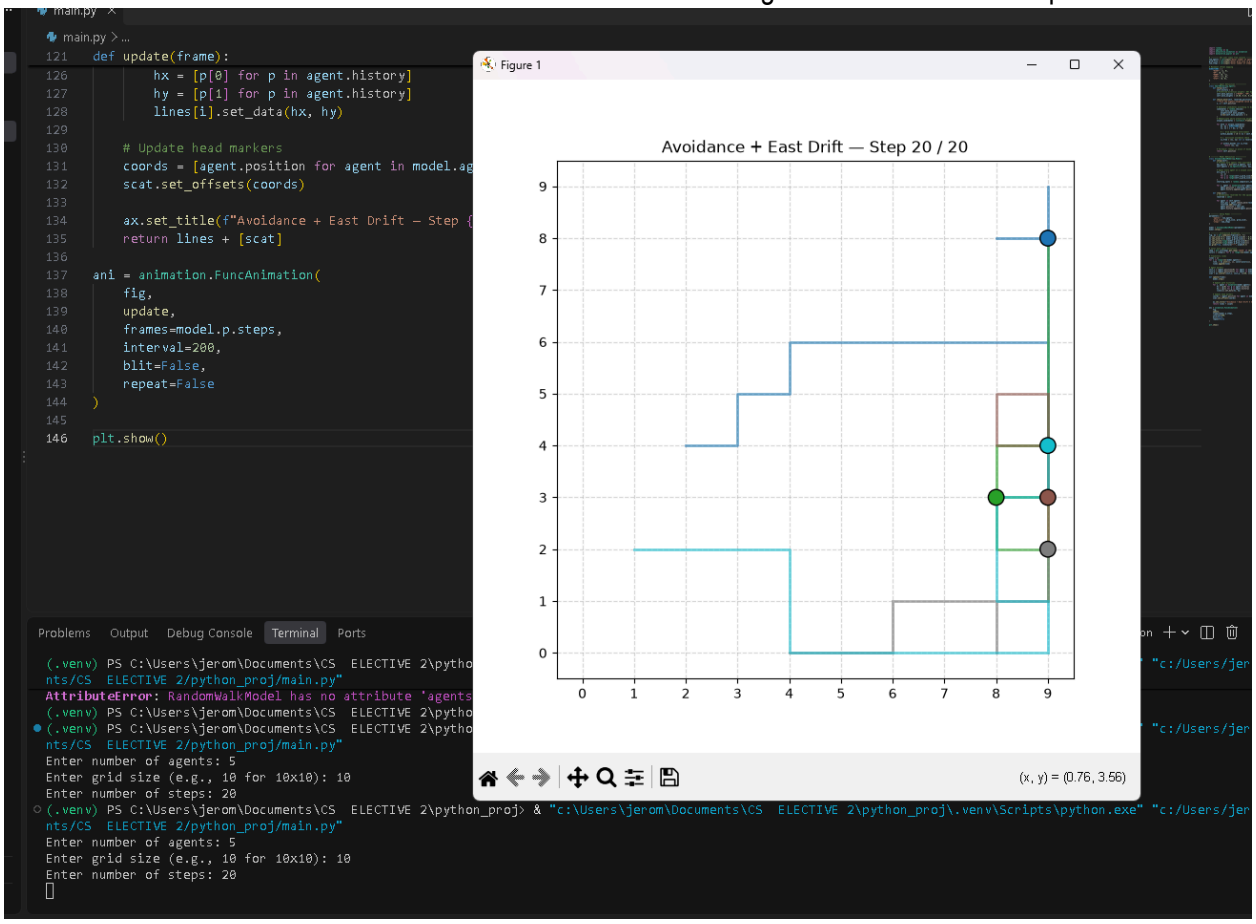
- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

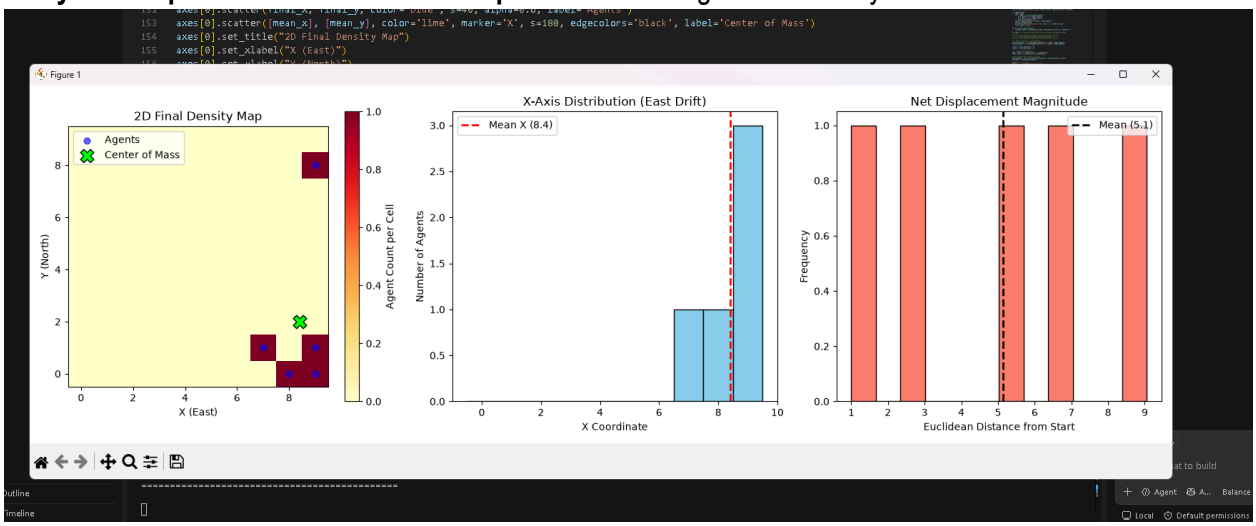
1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.



2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.



3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.



4.

5. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Simulation 1:

```
       FINAL DISTRIBUTION ANALYSIS
============================================
Total Agents Simulated  : 5
Grid Dimensions         : 10 x 10
Total Steps Executed    : 20
--------------------------------------------
Center of Mass (Mean)   : X = 8.40, Y = 2.00
Spatial Spread (Std Dev): X = 0.80, Y = 3.03
Mean Net Displacement   : 5.14 grid units
Agents at Right Wall (X=9): 3 / 5
============================================
```

Simulation 2:

```
nts/CS  ELECTIVE 2/python_proj/main.py
       FINAL DISTRIBUTION ANALYSIS
============================================
Total Agents Simulated  : 7
Grid Dimensions         : 6 x 6
Total Steps Executed    : 15
--------------------------------------------
Center of Mass (Mean)   : X = 4.43, Y = 2.14
Spatial Spread (Std Dev): X = 0.73, Y = 1.81
Mean Net Displacement   : 2.32 grid units
Agents at Right Wall (X=5): 4 / 7
============================================
```

Modified (Updated Code - Laboratory Tasks) ⇩

```
import random
import agentpy as ap
import matplotlib.animation as animation
import matplotlib.pyplot as plt
import numpy as np

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

DIRECTIONS = {
    "RIGHT": (1, 0),
```

```
    "UP": (0, 1),
    "DOWN": (0, -1),
    "LEFT": (-1, 0),
    "STAY": (0, 0),
}

# -------- Agent Definition -----------
class SmartWalker(ap.Agent):
    def setup(self):
        self.history = []
        self.move_options = ["RIGHT", "UP", "DOWN", "LEFT", "STAY"]
        self.move_weights = [0.60, 0.15, 0.15, 0.05, 0.05]

    def choose_move(self, reserved_positions):
        x, y = self.position
        candidates = random.choices(
            self.move_options,
            weights=self.move_weights,
            k=len(self.move_options) * 2
        )
        unique_candidates = list(dict.fromkeys(candidates))

        for move in unique_candidates:
            dx, dy = DIRECTIONS[move]
            nx, ny = x + dx, y + dy

             within_bounds = (0 <= nx < self.model.p.grid_size[0]) and (0
<= ny < self.model.p.grid_size[1])
             is_free = (nx, ny) not in reserved_positions or (nx == x and
ny == y)

            if within_bounds and is_free:
                return (nx, ny)

        return self.position

# -------- Model Definition -----------
```

```
class AvoidanceWalkModel(ap.Model):
    def setup(self):
                max_agents  =  min(self.p.agents,  self.p.grid_size[0]  *
self.p.grid_size[1])
        self.agents = ap.AgentList(self, max_agents, SmartWalker)

         all_cells = [(x, y) for x in range(self.p.grid_size[0]) for y in
range(self.p.grid_size[1])]
        starting_spots = random.sample(all_cells, max_agents)

        for i, agent in enumerate(self.agents):
            agent.position = starting_spots[i]
            agent.history.append(agent.position)

    def step(self):
        reserved = set()
        for agent in self.agents:
            next_pos = agent.choose_move(reserved)
            reserved.add(next_pos)
            agent.position = next_pos
            agent.history.append(agent.position)

# -------- Setup Model -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

model = AvoidanceWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig_anim, ax_anim = plt.subplots(figsize=(6, 6))
ax_anim.set_xlim(-0.5, model.p.grid_size[0] - 0.5)
ax_anim.set_ylim(-0.5, model.p.grid_size[1] - 0.5)
ax_anim.set_xticks(range(model.p.grid_size[0]))
```

```
ax_anim.set_yticks(range(model.p.grid_size[1]))
ax_anim.grid(True, linestyle="--", alpha=0.5)

cmap = plt.colormaps.get_cmap('tab10' if len(model.agents) <= 10 else
'viridis').resampled(len(model.agents))
colors = [cmap(i) for i in range(len(model.agents))]

lines = [ax_anim.plot([], [], color=colors[i], alpha=0.5,
linewidth=1.5)[0] for i in range(len(model.agents))]
init_x = [agent.position[0] for agent in model.agents]
init_y = [agent.position[1] for agent in model.agents]
scat = ax_anim.scatter(init_x, init_y, s=120, c=colors,
edgecolors='black', zorder=3)

def update(frame):
    model.step()
    for i, agent in enumerate(model.agents):
        hx = [p[0] for p in agent.history]
        hy = [p[1] for p in agent.history]
        lines[i].set_data(hx, hy)
    coords = [agent.position for agent in model.agents]
    scat.set_offsets(coords)
    ax_anim.set_title(f"Simulation — Step {frame + 1} / {model.p.steps}")
    return lines + [scat]

ani = animation.FuncAnimation(
      fig_anim, update, frames=model.p.steps, interval=150, blit=False,
repeat=False
)
plt.show()   # Close this window when finished to trigger the analysis
phase

# ============================================================
# -------- POST-SIMULATION: RECORD & ANALYZE POSITIONS ---------
# ============================================================

# 1. Extract Initial vs. Final Positions
```

```
initial_positions = np.array([agent.history[0] for agent in model.agents])
final_positions = np.array([agent.history[-1] for agent in model.agents])

final_x = final_positions[:, 0]
final_y = final_positions[:, 1]

# 2. Statistical Computations
mean_x, mean_y = np.mean(final_x), np.mean(final_y)
std_x, std_y = np.std(final_x), np.std(final_y)

# Net Euclidean displacement per agent
displacements = np.linalg.norm(final_positions - initial_positions,
axis=1)
mean_disp = np.mean(displacements)

print("\n" + "=" * 45)
print("       FINAL DISTRIBUTION ANALYSIS")
print("=" * 45)
print(f"Total Agents Simulated  : {len(model.agents)}")
print(f"Grid Dimensions         : {grid_size} x {grid_size}")
print(f"Total Steps Executed    : {num_steps}")
print("-" * 45)
print(f"Center of Mass (Mean)   : X = {mean_x:.2f}, Y = {mean_y:.2f}")
print(f"Spatial Spread (Std Dev): X = {std_x:.2f}, Y = {std_y:.2f}")
print(f"Mean Net Displacement   : {mean_disp:.2f} grid units")
print(f"Agents at Right Wall (X={grid_size-1}): {np.sum(final_x ==
grid_size - 1)} / {len(model.agents)}")
print("=" * 45 + "\n")

# 3. Distribution Visualizations
fig_analysis, axes = plt.subplots(1, 3, figsize=(16, 5))

# (A) Final 2D Density Heatmap
density_grid = np.zeros((grid_size, grid_size), dtype=int)
for x, y in final_positions:
    density_grid[y, x] += 1  # Rows: Y, Cols: X
```

```
im = axes[0].imshow(density_grid, origin='lower', cmap='YlOrRd',
extent=[-0.5, grid_size - 0.5, -0.5, grid_size - 0.5])
axes[0].scatter(final_x, final_y, color='blue', s=40, alpha=0.6,
label='Agents')
axes[0].scatter([mean_x], [mean_y], color='lime', marker='X', s=180,
edgecolors='black', label='Center of Mass')
axes[0].set_title("2D Final Density Map")
axes[0].set_xlabel("X (East)")
axes[0].set_ylabel("Y (North)")
axes[0].legend(loc="upper left")
plt.colorbar(im, ax=axes[0], label="Agent Count per Cell")

# (B) Marginal X Distribution (Direction of Bias)
axes[1].hist(final_x, bins=np.arange(-0.5, grid_size + 0.5, 1),
color='skyblue', edgecolor='black')
axes[1].axvline(mean_x, color='red', linestyle='--', linewidth=2,
label=f'Mean X ({mean_x:.1f})')
axes[1].set_title("X-Axis Distribution (East Drift)")
axes[1].set_xlabel("X Coordinate")
axes[1].set_ylabel("Number of Agents")
axes[1].legend()

# (C) Net Displacement Distribution
axes[2].hist(displacements, bins=12, color='salmon', edgecolor='black')
axes[2].axvline(mean_disp, color='black', linestyle='--', linewidth=2,
label=f'Mean ({mean_disp:.1f})')
axes[2].set_title("Net Displacement Magnitude")
axes[2].set_xlabel("Euclidean Distance from Start")
axes[2].set_ylabel("Frequency")
axes[2].legend()

plt.tight_layout()
plt.show()
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
   The system experiences phase transitions from free-flow to spatial jamming:
2. How does increasing the number of steps affect the **spread of agents**?
   The longer the simulation runs, the further agents wander away from their starting points and spread out across the whole space.
3. Do the agents ever move outside the grid? Why or why not?
   No, the agents never move outside the grid because of the agent decision rule.
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?
   I think this simulation would help in simulating evacuation planning for buildings or establishments to find the best strategies to move a crowd of people in a contained space.
5.

Answer the Questions ⇩